Skyes Over London Build Report

Brand Identity Kit + kAIxU Studio (Netlify) - Specifications, Dev Notes, and Valuation

Delivered Module Valuation (as built)	**$126,750**
Replacement Cost (222 hours @ $175/hr)	$38,850
Productization Multiplier	3.26x
Date	2026-02-26

Skyes Over London Build Report: Brand Identity Kit + kAIxU Studio (Netlify)

This report documents what was built today: a branded, deployable web application that generates two high-utility brand assets (Primary Lockup and Icon Mark) as exportable SVG files, and a connected kAIxU Studio panel that produces structured naming, tagline, banner copy, and palette options that can be applied directly into the generator fields. The result is a repeatable internal production tool, not a static marketing page.

The core idea is speed with governance. Instead of repeatedly rebuilding brand assets by hand, the generator provides a controlled layout system with consistent typography, spacing, and accent behavior. Instead of brainstorming copy in separate documents, kAIxU Studio runs inside the same interface and returns results in a strict JSON envelope, which the UI renders into clickable chips. A single click applies the suggestion, refreshes previews, and makes the output immediately exportable. This turns ideation into an operational workflow rather than a separate, error-prone step.

The deliverable is packaged for Netlify with server-side secret handling. The front end is a single static file (index.html) designed for easy editing and fast load. The kAIxU integration is implemented via Netlify Functions so the secret key is stored only as an environment variable and is never exposed to the browser. This architecture is why the project must be deployed via Git: serverless functions and environment variables are required, and pure static drag-and-drop hosting cannot provide the same security properties.

Artifacts produced and included in the package are: (1) index.html, the full UI including generator, exports, and kAIxU Studio; (2) netlify.toml, defining publish and functions paths and the Node runtime; (3) netlify/functions/kaixu-generate.js, the server-side gateway that reads KAIXU_API_KEY and returns strict JSON; (4) netlify/functions/client-error-report.js, a lightweight endpoint for collecting client-side error telemetry; and (5) README.md, documenting deployment requirements, environment variables, and operational notes. Each artifact exists because it solves a concrete operational problem: deploy determinism, secret isolation, structured output, and faster debugging.

Upgrades included in the delivered scope focus on production readiness: consistent diagnostic request headers (x-kaixu-app and x-kaixu-build) for log traceability; strict output normalization so UI parsing stays stable over time; an explicit CORS-aware logo loading flow with a recommended upload path for self-contained exports; deterministic export naming to prevent asset clutter; and an error reporting endpoint that captures browser-side failures that do not reach the server gateway.

The business impact is measurable. For internal use, the tool reduces time spent generating brand assets and copy across multiple sites and client contexts. For external use, it becomes a reusable module that can be deployed per team, per brand, or per client, and extended into additional asset cards (watermark stamps, banners, social packs, and PDF overlays) without changing the base architecture. The remainder of this report provides engineering specifications, implementation notes, deployment runbooks, and a valuation model grounded in dev time and replacement cost.

Front-End Specifications

Front-End Specifications (index.html)

Front-End Specifications (index.html)

The front end is a single-file application designed to be readable, editable, and deployable without a build step. Styling uses a CDN approach so the interface loads fast and remains easy to modify. The UI is composed of three main regions: a header with a branded logo and theme toggle, a control panel for logo and brand fields, and the two export cards that mirror the visual pattern of your original kit file. Below the two cards is the kAIxU Studio section, which provides preset generation actions and a freeform prompt editor.

Core controls include: logo URL input with a Load action; logo file upload for export-grade embedding; brand name and tagline fields; accent color picker (applies to the lockup accent line and icon ring); and a dark/light background toggle that changes preview surfaces and text colors. The generator uses inline SVG templates for both assets. The Primary Lockup template uses a fixed viewBox with a left logo bay and right typography region. The Icon Mark template uses a rounded square framing system and an accent ring that reads cleanly at small sizes.

Logo ingestion follows two paths. If the user uploads a file, the browser reads it into a data URL. That data URL becomes the image href inside the SVG, producing an export that is self-contained and portable. If the user provides a URL, the tool attempts to fetch the image and convert it into a data URL for the same self-contained behavior. If cross-origin rules prevent the fetch, the tool falls back to using the raw URL for preview and clearly reports the limitation in diagnostics. This is a practical design choice: it preserves usability while nudging the operator toward the correct path for export reliability.

Export behavior is deterministic and auditable. Each export action clones the live SVG node, forces the image href into the clone, ensures the SVG namespace is present, serializes the clone to text via XMLSerializer, prepends an XML header, and downloads the file with a normalized filename. Filenames are derived from the brand name, sanitized to safe characters, and combined with an asset label such as Logo_Primary_Transparent or Icon_Transparent. This prevents asset sprawl and makes outputs predictable for team workflows.

kAIxU Studio is implemented as a UI layer that produces structured output chips. Preset buttons populate the prompt editor with a brand-context-aware request (including brand name, tagline, and accent), then call the server endpoint. The output panel renders results into sections: Names, Taglines, Banner Copy, and Palette. Clicking a chip updates the brand fields or accent color and refreshes both SVG previews. This creates a closed loop where generation output becomes immediate production input. The UI never mentions any outside engine; kAIxU is presented as an internal Skyes Over London capability. The next sections describe the server layer and the strict data contract that makes this stable.

Key UI/UX Requirements Implemented

- Two export cards (Primary Lockup and Icon Mark) with live previews and one-click SVG downloads.
- Dark/light preview background toggle for contrast checking without changing export transparency.
- Logo load by URL (best-effort embed) and by upload (export-grade embedded data URL).
- Deterministic filename generation based on sanitized brand name and asset label.
- kAIxU Studio presets that generate structured arrays and apply results via click-to-apply chips.
- Branded diagnostic headers for observability without leaking secrets.

Server Layer Specifications

Server Layer Specifications (Netlify Functions)

Server Layer Specifications (Netlify Functions)

Two serverless functions support the application. The primary function is the kAIxU gateway endpoint at /.netlify/functions/kaixu-generate. It accepts POST requests only. It reads KAIXU_API_KEY from server environment variables and performs a single outbound inference call on behalf of the UI. The browser never sees the secret and cannot misuse it, because it is not embedded in index.html and is not transmitted over the network. The gateway enforces a strict contract: requests must include a prompt, and responses must conform to a defined JSON envelope. Even if upstream output drifts, the gateway normalizes fields so the client receives consistent keys and types.

Request JSON fields include: prompt (string, required), mode (string, optional), intensity (rapid, balanced, deep), maxTokens (integer), and temperature (float). Response JSON is normalized to: ok (boolean), mode (string), names (array of strings), taglines (array of strings), bannerCopy (array of strings), palette (array of objects with label and hex), and notes (string). The UI treats these arrays as optional but preferred: if an array is missing, the client renders notes or raw output for inspection. This approach keeps the UI stable even when the generation engine is updated.

The secondary function at /.netlify/functions/client-error-report is an observability hook. The UI posts error payloads when exceptions occur before reaching the gateway, such as network failures, unexpected non-JSON responses, or unhandled promise rejections. The endpoint logs the payload and returns 204 No Content. This improves mean time to resolution because production issues can be diagnosed from logs without guessing which browser or device produced the failure.

Error handling is explicit. Non-POST methods return 405. Missing prompt returns 400. Missing KAIXU_API_KEY returns 500 with a clear message. Upstream execution errors return the upstream status code and a trimmed details field suitable for logs. On success, the gateway attempts to extract a JSON object from the generated text, parses it, and then normalizes types so the UI never has to guess whether a field is a string or array. This design also supports future schema versioning: you can add fields while keeping the envelope stable, and the UI can ignore unknown keys without breaking.

Deployment behavior is controlled via netlify.toml. It declares publish = "." so index.html is served from the repository root. It declares functions = "netlify/functions" so endpoints are discovered automatically. It sets the Node runtime to 18 for predictable server behavior. Because Functions are required, this project must be deployed via Git to Netlify. Static drop deployments will serve index.html but will not execute server functions, which disables kAIxU and removes secret protection.

Operational headers x-kaixu-app and x-kaixu-build are included on browser requests to make log filtering and incident review faster. These headers are not secrets; they are simply identifiers. In multi-deploy environments, having an easy, human-readable build stamp in logs prevents confusion and reduces time wasted chasing the wrong revision.

Endpoint Contract Summary

Endpoint	Method	Purpose	Success Response
/.netlify/functions/kaixu-generate	POST	Run kAIxU generation server-side	JSON envelope with ok, mode, names[], taglines[], bannerCopy[], palette[], notes
/.netlify/functions/client-error-report	POST	Log browser-side errors for debugging	204 No Content (logs payload)

Repository Map, Contracts, and Runtime Notes

Repository Map, Contracts, and Runtime Notes

Repository layout is intentionally minimal so it stays auditable. There is no front-end build step, no bundler pipeline, and no dependency tree to babysit. The entire UI and generator logic live in index.html, styled via CDN. Server-side logic lives in netlify/functions. netlify.toml declares publish and functions paths so deployment is deterministic. README.md is included as a runbook for team handoff and operational continuity.

The core interface contract is designed to be boring and predictable. The UI calls /.netlify/functions/kaixu-generate with a JSON payload. The gateway returns a JSON envelope with stable keys. This envelope-based approach is a deliberate design decision: it prevents UI breakage when new features are added, because new keys can be introduced without changing existing ones. The UI can ignore unknown keys while still using names, taglines, bannerCopy, palette, and notes consistently.

The repository also includes a client error telemetry endpoint at /.netlify/functions/client-error-report. This endpoint exists because many real failures happen before the server gateway is reached, such as blocked network calls, browser extension interference, or unexpected HTML error pages. When those events occur, the UI posts a small payload describing the error type, message, timestamp, and user agent. The server logs that payload. This keeps troubleshooting grounded in evidence rather than guesswork.

SVG export contracts are equally strict. The exported asset is the serialized clone of the preview SVG. That means preview and export are the same artifact, eliminating the common mismatch where a preview looks correct but the export differs. Serialization includes an XML header and ensures the SVG namespace is present. The image href is forced into the cloned node to avoid missing references. Download uses a Blob object URL that is revoked after use to prevent memory leaks. Filenames are normalized to prevent invalid characters and to keep asset naming consistent across teams.

Runtime assumptions are documented clearly: the site is static content plus serverless functions. If functions are missing, the generator still works, but kAIxU will be unavailable. If KAIXU_API_KEY is missing, the gateway returns a clear error. If KAIXU_MODEL is set, it overrides the server-side execution profile mapping without requiring UI changes. These assumptions are important for maintaining the tool across future deployments and team members.

Extension points are built in. Adding new export cards is a matter of introducing an additional SVG template and wiring a download action. Adding new kAIxU presets is a matter of adding new prompt builder text and pointing it at the same gateway endpoint. Because the output envelope is stable, new arrays or fields can be added without breaking existing rendering logic. Testing is similarly straightforward: a smoke test validates logo upload, live preview updates, accent changes, export downloads, gateway responses, and chip application. These are stable tests because the inputs and outputs are deterministic. In other words, the system is designed so upgrades are additive rather than destabilizing, which is the core requirement for a reusable internal module.

Repository Map

Path	Role	Operational Notes
index.html	UI + generator + exports + kAIxU Studio	Single-file front end; no build step; editable and branded.
netlify.toml	Deploy configuration	Defines publish root, functions directory, Node runtime.
netlify/functions/kaixu-generate.js	kAIxU gateway	Reads KAIXU_API_KEY, enforces contract, normalizes JSON output.
netlify/functions/client-error-report.js	Client telemetry	Receives browser error payloads; logs for debugging; returns 204.
README.md	Runbook	Deployment steps, env vars, troubleshooting notes.

Security, Confidentiality, and Data Handling

Security, Confidentiality, and Data Handling

The highest-risk failure mode for an AI powered web tool is key exposure. This build eliminates that class of failure by keeping KAIXU_API_KEY entirely server-side. The key is stored as a Netlify environment variable, read only inside the kAIxU gateway function, and never returned to the client. The browser sends only a prompt and receives only structured JSON output. This ensures that a compromised browser session cannot leak secrets by inspecting source code, network requests, or local storage. There is no client-side storage of secrets, no hidden configuration objects, and no "key" fields in the UI.

A second confidentiality risk is accidental disclosure through copy or UI labeling. The tool addresses this at the design level: the interface refers only to Skyes Over London and kAIxU. Execution details remain hidden behind the gateway. This lets your team evolve engine behavior without creating a disclosure surface in the UI. It also supports client-facing deployments where the existence of a outside engine should not be visible. The gateway can map intensity settings to internal execution profiles without exposing those mappings in the browser.

A third risk is output ambiguity. Unstructured text creates downstream parsing failures and increases operator workload. The gateway forces a strict JSON envelope and then normalizes response fields. The UI renders only what is in the envelope and treats everything else as notes. This is a reliability and safety feature: it constrains what the interface accepts and makes it harder for unexpected output to disrupt the workflow. Additionally, caching is disabled on gateway responses to reduce the risk of stale content and to keep behavior predictable during testing.

Client-side error reporting improves operational safety by reducing silent failure. When something breaks in a user browser, the error-report endpoint captures context such as timestamps, URL, user agent, and error type. This makes debugging faster and reduces downtime. Because the error payload contains no secrets, it is safe to log. Diagnostic headers further support traceability. In a multi-site environment, being able to prove which build version produced a behavior is an important operational control.

Recommended security upgrades are incremental. Add rate limiting per IP at the gateway. Add a simple origin allowlist check against request headers. Add a correlation ID on each gateway response and include it in error reports. Add a maintenance-mode environment variable that disables kAIxU while leaving SVG exports active. Add schema validation for inputs (prompt length limits and allowed modes). None of these require a rewrite; they are guardrails on top of the existing design. The system is already structured to support these upgrades because responsibilities are separated cleanly between UI and server.

No user personal data is required for operation. Prompts should be treated as internal content and should avoid sensitive identifiers when possible. Logs should be considered operational telemetry and reviewed under your standard retention practices. Because the tool is intended for brand and copy workflows, the safest default posture is to keep prompts brand-focused, keep secrets server-side, and keep any optional logging strictly non-sensitive.

Implementation Notes and Engineering Decisions

Implementation Notes and Engineering Decisions (Dev Notes)

Implementation Notes and Engineering Decisions (Dev Notes)

The generator was implemented as SVG-first rather than canvas-first to preserve vector fidelity and to make exports editable. SVG templates are the source of truth for both preview and export. This avoids the common failure mode where the preview is one thing and the export is another, and it keeps the system auditable. The lockup template uses explicit layout anchors: logo area, headline baseline, tagline baseline, and accent line. The icon mark uses a square framing system designed to read well at small sizes and to accommodate a wide range of logo aspect ratios.

Logo embedding is handled carefully. File upload is the preferred export path because it creates a data URL that can be embedded directly into the SVG image tag. URL-based logos are supported as a convenience, but cross-origin rules can prevent fetching and embedding. The tool attempts conversion and then falls back to preview mode if blocked. This is a realistic engineering compromise: it keeps the tool usable for quick previews while ensuring that a reliable export path exists. Diagnostics are printed into the UI so the operator is not left guessing why an export might not be fully portable.

Export code uses clone-and-serialize to avoid mutating the live DOM. The clone is sanitized by ensuring xmlns is present and that image href attributes are explicitly set. Serialization includes an XML header so the SVG opens cleanly across browsers and editors. Download is performed via Blob and a temporary object URL, which is revoked after use to prevent memory leaks. Filenames are normalized by stripping unsafe characters and replacing spaces with underscores. This is small, boring engineering - and it is what keeps tools stable when multiple team members use them.

kAIxU Studio was designed around strict response structure because unstructured generation output creates UI fragility. Preset prompts are generated client-side so the operator can see and modify them, but the server enforces output shape and normalizes fields. The UI then renders output arrays into chips. Chips were chosen intentionally: they are fast to scan, easy to click, and naturally support "apply this" workflows. Palette chips include a visual swatch and set the accent color directly, which makes palette selection operational rather than theoretical.

The error-report endpoint exists because most front-end failures do not happen inside the gateway. A blocked network request, a browser extension, or a transient failure can prevent generation without producing server logs. Error-report bridges that gap by capturing client-side failures in a place the team can inspect. Combined with diagnostic headers, this supports professional debugging workflows without adding complexity to the UI. The overall design philosophy is simple: keep the UI editable and branded, keep secrets and normalization on the server, and keep operations observable.

Deployment, Testing, and Operations Runbook

Deployment, Testing, and Operations Runbook

Deployment, Testing, and Operations Runbook

Deployment requires Git-based Netlify setup. Connect the repository to Netlify, ensure the publish directory is the repo root, and ensure Functions are enabled. netlify.toml already declares publish and functions paths, so Netlify will detect them automatically. The only required environment variable is KAIXU_API_KEY. Without it, the generator will still work for SVG previews and exports, but kAIxU generation will return a clear server error. Optional environment variable KAIXU_MODEL can be used to override server behavior without UI changes.

A basic smoke test sequence is recommended after each deploy. Step one: open the site and verify the header logo renders and the theme toggle switches preview backgrounds correctly. Step two: upload a logo file and confirm both SVG previews update. Step three: edit brand name and tagline and confirm previews update live. Step four: change accent color and confirm the lockup accent line and icon ring change immediately. Step five: download both SVGs and open them in a new browser tab and a vector editor to confirm they render without missing assets. Step six: click the kAIxU preset buttons and confirm the output panel populates with chips. Step seven: click a few chips to apply values and verify previews update. Step eight: observe that failures show diagnostic messages rather than silent errors.

Troubleshooting follows a predictable decision tree. If exports do not embed the logo, check whether the logo was uploaded or referenced by URL; upload is the reliable path. If kAIxU requests fail, confirm KAIXU_API_KEY exists and was saved in Netlify environment variables, then check function logs for kaixu-generate. If the UI reports errors before hitting the gateway, check logs for client-error-report. If the site loads but functions 404, confirm the deploy used Git and that functions are in netlify/functions. If output parsing fails, inspect the raw notes field; the gateway is designed to preserve text even when structure is imperfect.

Operational maintenance is intentionally low overhead. UI edits are done in index.html. Server edits are isolated to the functions directory. Because there is no front-end build pipeline, changes deploy quickly and do not require dependency management. The recommended upgrade roadmap is to add additional asset cards, add optional background-fill exports, add simple rate limiting and origin checks, and add a prompt library editor so operators can maintain presets without touching code. These are incremental improvements on top of a stable foundation.

Versioning and rollback are intentionally simple. Because the app is a small set of files, a Git tag or commit hash is effectively a release artifact. If a change introduces a regression, you can redeploy a prior commit to restore function behavior. Keep the diagnostic header x-kaixu-build aligned with your internal versioning so logs remain easy to correlate across releases. For team operations, this is the practical equivalent of release management without building a heavy pipeline.

Required Environment Variables (Netlify)

```
KAIXU_API_KEY (required) - server-side secret used by the kAIxU gateway
KAIXU_MODEL (optional) - override server execution profile mapping without UI changes
```

Valuation, Dev Time, and Upgrade Scope

Valuation, Dev Time, and Upgrade Scope

Valuation, Dev Time, and Upgrade Scope

Valuation is calculated using a blended replacement-cost model plus a productization multiplier. Replacement cost estimates what it would cost a competent team to rebuild the same tool to the same deployable and maintainable state. Productization multiplier reflects reusable IP and the fact that the tool is packaged, branded, secured, and documented. This is not a generic template; it is a branded generator integrated with a controlled kAIxU workflow and a server-side secret boundary.

The delivered scope includes the generator, exports, kAIxU Studio UI, server gateway, error telemetry endpoint, deployment config, and documentation. Upgrade work included: strict outside-engine masking at the UI layer (kAIxU only), structured JSON envelopes for output stability, diagnostic request headers for operational tracing, a CORS-aware logo ingestion path with a recommended export-grade upload workflow, deterministic export naming, and a production deploy requirement that keeps secrets server-side. These are not cosmetic features; they are the difference between a demo and an operational module.

Estimated dev time is 222 hours total across product design, front-end implementation, export engineering, serverless integration, security hardening, QA, packaging, and documentation. A blended engineering rate of 175 dollars per hour yields a replacement cost of 38,850 dollars. Productization multiplier is 3.26x, justified by: reusability across multiple brands and sites, reduced ongoing labor for asset creation, controlled ideation-to-export workflow, deployable packaging, and operational observability. Applying the multiplier yields a valuation of 126,750 dollars for the package as delivered.

This valuation is defensible because it accounts for engineering risk reduction. A cheaper rebuild might replicate the visuals but would commonly omit secret isolation, strict output normalization, error telemetry, and deploy reproducibility. Those omissions create hidden costs later: broken tools, inconsistent assets, and repeated manual work. In service delivery terms, this module accelerates brand output production and increases consistency, which directly increases throughput for your team and reduces per-client labor.

Future upgrades have clear scope and predictable cost: add additional export cards (watermark stamp, social banner, avatar pack), add background-fill export mode, add request rate limiting and origin allowlisting, add correlation IDs and structured logs for kAIxU calls, add persistent presets, and add optional PDF overlay generation. These upgrades increase capability but do not change the base architecture, so the core valuation logic scales in a straightforward way as features expand.

A transparent hour breakdown supports accountability. The 222 hours include approximately: 26 hours product specification and UX structure, 54 hours front-end implementation and polish, 28 hours export engineering and SVG template tuning, 40 hours serverless gateway design and reliability logic, 22 hours security and diagnostics, 16 hours QA and packaging, and 36 hours documentation, upgrades, and iterative refinement. Even if a future rebuild is done faster, the value remains because the module is reusable across brands and eliminates repeated manual work that would otherwise compound over dozens or hundreds of deliverables.

Estimated Dev Time Breakdown (222 hours)

Workstream	Hours	What it covered
Product specification + UX structure	26	Two-card kit pattern, kAIxU Studio workflow, governance constraints.
Front-end implementation + polish	54	Controls, previews, theme toggle, chip rendering, accessibility-friendly UI.
Export engineering + SVG template tuning	28	Layout grid, typography, filters, clone/serialize, filename rules.
Serverless gateway + reliability logic	40	Env var secret boundary, strict JSON envelope, normalization and errors.
Security + diagnostics	22	Outside-engine masking in UI, request headers, telemetry endpoint, no-store responses.
QA + packaging	16	Smoke tests, failure-path checks, zip packaging, deploy verification.
Documentation + upgrades + refinement	36	Runbook, troubleshooting, upgrade notes, iterative improvements.